Universität Bonn

Mitschrift zur Vorlesung

Grundlagen der algorithmischen Geometrie

gehalten von

Dr. Herman Haverkort,
PD. Dr. Elmar Langetepe

geTEXed von

Daniel Emse

Sommer 2026

Anmerkungen und Korrekturen

Falls du Anmerkungen/Korrekturen hast, melde dich bei mir (`daniel.emse@uni-bonn.de`).

Inhaltsverzeichnis

Vorlesung **1**

2 Das Sweep-Verfahren **2**
2.1 Einführung 2
2.2 Sweep im Eindimensionalen 2
2.2.1 Das Maximum einer Menge von Objekten 2
2.2.2 Das dichteste Paar in einer Menge von Zahlen 2
2.2.3 Die maximale Teilsumme 3
2.3 Sweep in der Ebene 3
2.3.1 Das dichteste Punktepaar in der Ebene 3
2.3.2 Schnittpunkte von Strecken 5
2.3.3 Einschub – Untere Schranken 7

Anhang **A**

Liste der Definitionen **B**

Liste der Aussagen **C**

Liste der Algorithmen **D**

Liste der Abkürzungen **E**

Index **F**

Vorlesung

Kapitel 2

Das Sweep-Verfahren

2.1 Einführung

[14.04.2026, Vorlesung 1]

Das *Sweep*-Verfahren ist ähnlich vielseitig wie *Divide and Conquer*, also ein hilfreiches Pardigma. Divide and Conquer ist beispielsweise sinnvoll, wenn die erwarteten Kosten des Ergebnisses klein sind. Beispielsweise erhalten wir $T(n) \leq 2 \cdot T(n/2) + c(\text{merge})$.

Sweep ist ein geometrisches Verfahren, bei dem wir lokal eine Dimensionsreduktion von einem n-dimensionalen Problem auf ein $(n-1)$-dimensionales Problem verwandeln, indem wir die Ebene mit einer *Sweepline* ausfegen, indem wir gemäß einer vorher angefertigten Sortierung fegen. Dabei folgt die Korrektheit des Verfahrens über eine Invariante nahe der Sweepline.

2.2 Sweep im Eindimensionalen

2.2.1 Das Maximum einer Menge von Objekten

Problem 2.1: Find maximum

Input: $Q = [q_1, \ldots, q_n] \in \mathbb{R}^n$
Output: $q_i \in Q$ mit $q_i \geq q_j \forall j = 1, \ldots, n$

Hierfür haben wir folgenden optimalen Algorithmus mit Laufzeit $\Theta(n)$. Dabei sweepen wir über die Indizes, erhalten also lokal ein 0-dimensionales Problem

Algorithmus 2.2 FindMax

Input: Wie in Problem 2.1
Output: Wie in Problem 2.1

```
1: MaxSoFar ← q[1]
2: for j = 2 to n do do
3:     if MaxSoFar < q[j] then
4:         MaxSoFar ← q[j]
5: return MaxSoFar
```

2.2.2 Das dichteste Paar in einer Menge von Zahlen

Problem 2.3: Closest pair in einer Dimension

Input: $x_1, \ldots, x_n \in \mathbb{R}$
Output: $i \neq j \in \{1, \ldots, n\}$ sodass $|x_i - x_j| = \min_{\ell \neq k} |x_\ell - x_k|$ für $\ell, k \in \{1, \ldots, n\}$

Wir sortieren die Zahlen aufsteigend, sodass wir ein neues Array $x'_1 \leq \cdots \leq x'_n \in \mathbb{R}$ bekommen (im Mehrdimensionalen sortieren wir sie nach einer bestimmten Koordinate). Das braucht aber natürlich bereits $\mathcal{O}(n \log(n))$ Zeit.

Algorithmus 2.4 FINDCLOSESTPAIR-1D

Input: Wie in Problem 2.3, aber als aufsteigend sortiertes Array x'
Output: Wie in Problem 2.3

```
1: MinDistSoFar ← x'[2] − x'[1]
2: ClosPos ← 2
3: for j = 3 to n do do
4:     if MinDistSoFar > x'[j] − x'[j − 1] then
5:         MinDistSoFar ← x'[j] − x'[j − 1]
6:         ClosPos ← j
7: return x'[ClosPos − 1], x'[ClosPos]
```

Korollar 2.5. *Die Berechnung des CLOSEST PAIRS von n reellen Zahlen benötigt Laufzeit $\mathcal{O}(n \log(n))$.*

2.2.3 Die maximale Teilsumme

Problem 2.6: Maximum subvector

Input: Array $V \in \mathbb{R}^n$
Output: Finde $i \leq j$, sodass $V[i] + V[i+1] + \cdots + V[j] = \max_{\ell \leq k} V[\ell] + \cdots + V[k]$, mit $i, j, \ell, k \in \{1, \ldots, n\}$

Algorithmus 2.7 FINDMAXIMUMSUBVECTOR

```
1: MaxSoFar ← 0
2: MaxEndingHere ← 0
3: for j = 1 to n do do
4:     MaxEndingHere ← max(0, MaxEndingHere + V[j])
5:     MaxSoFar ← max(MaxSoFar, MaxEndingHere)
6: return MaxSoFar
```

Satz 2.8: FindMaximumSubvector

In einer Folge von n Zahlen die maximale Teilsumme konsekutiver Zahlen zu bestimmen, hat die Zeitkomplexität $\Theta(n)$.

2.3 Sweep in der Ebene

Im Allgemeinen ist $|x| \coloneqq \|x\|_2$ die Euklidische Norm des Vektor x.

2.3.1 Das dichteste Punktepaar in der Ebene

Problem 2.9: Closest pair in zwei Dimensionen

Input: $x_1, \ldots, x_n \in \mathbb{R}^2$
Output: $i \neq j \in \{1, \ldots, n\}$ sodass $|x_i - x_j| = \min_{\ell \neq k} |x_\ell - x_k|$ für $\ell, k \in \{1, \ldots, n\}$

Wir führen zum Verwalten ein Verzeichnis ein, in dem zu jedem Zeitpunkt den Status der *Sweep Line* tracked. Dieses heißt *Sweep-Status-Struktur (SSS)*. Übergeordnet führen wir eine *Ereignisstruktur (ES)* ein, in dieser sind einfach alle Punkte nach ihrer x-Koordinate sortiert. Die SSS teilt diese in Streifen der Breite *MinSoFar* ein. Erreicht ein neuer Punkt solch einen Streifen, testen wir unter allen Punkten im Streifen, welche davon den geringsten Abstand haben. Das aktualisiert *MinSoFar* und der Streifen der SSS wird entsprechend schmaler.

Wir erhalten den folgenden Algorithmus:

Algorithmus 2.10 FindClosestPair-2D

Input: Array von n Punkten aus $\mathbb{R}^2$
Output: Zwei Elemente des Arrays mit dem geringsten euklidischen Abstand

```
1: Sortiere die n Punkte nach aufsteigenden x-Koordinaten und füge sie ins Array P ein. Das
   bildet die ES
2: Füge P[1], P[2] in SSS ein
3: MinSoFar ← |P[1] − P[2]|
4: links ← 1
5: rechts ← 3
6: while rechts ≤ n do
7:     if P[links].x + MinSoFar ≤ P[rechts].x then
8:         Entferne P[links] aus SSS
9:         links ← links + 1
10:    else
11:        MinSoFar ← MinDist(SSS, P[rechts], MinSoFar)
12:        Füge P[rechts] in SSS ein
13:        rechts ← rechts + 1
14: return MinSoFar
```

[14.04.2026, Vorlesung 1]
[16.04.2026, Vorlesung 2]

Zu spezifizieren ist noch MinDist. Wir müssen uns dafür Gedanken darüber machen, welche Datenstruktur wir am besten verwenden, um die SSS zu verwalten. Eine Liste hätte lineare Laufzeit, um die Nachbarn zu prüfen, das kann im schlimmsten Fall $\mathcal{O}(n^2)$ pro Ausführung von MinDistbenötigen kann.

Sei $r := P[\mathit{rechts}]$ und $M := \mathit{MinSoFar}$. Dann genügt es, für MinDist(SSS, r, M) nur die Punkte aus SSS anzuschauen, deren x-Koordinate in $[r.x - M, r.x + M]$ liegt. Folgende Operationen muss unsere SSS unterstützen:

- MinDist(SSS, r, M)
- Insert(p)
- Delete(p)

Wir verwenden als Datenstruktur einen balancierten Binärbaum über die y-Koordinaten als Schlüssel. Damit benötigen Insert und Delete jeweils $\mathcal{O}(\log(n))$, und Ausführung der Bereichsanfrage nach allen $p.y$ im Invervall $[r.x - M, r.x + M]$ benötigt Laufzeit $\mathcal{O}(\log(n) + k)$, wobei k die Größe der Antwort bedeutet.

Der betrachtete Streifen des Rechtecks um r hat einen Flächeninhalt von $2M^2 = 2M \cdot M$. Ein Punkt aus dem Rechteck deckt mit seinem Kreis mit Radius $\frac{M}{2}$ mindestens einen Viertelkreis als Flächeninhalt des Rechtecks ab, also mindestens $\pi\frac{M^2}{16}$. Wir erhalten mit

$$k \leq \frac{2M^2}{\pi\frac{M^2}{16}} \leq \frac{32}{\pi} < 11,$$

also ist $k \leq 10 \in \mathcal{O}(1)$ und die Bereichsanfrage benötigt auch nur $\mathcal{O}(\log(n))$ Zeit. Die Datenstruktur für SSS in diesem Fall ist also ein *Adelson-Velsky, Landis (AVL)-Baum*. Wir erhalten

folgendes Lemma:

Lemma 2.11: Höchstens 10 Punkte in Rechteck

Sei $M > 0$ und P eine Menge von Pnukten im $\mathbb{R}^2$, von denen je zwei den Abstand $\geq M$ haben. Ein Rechteck mit Kantenlängen M und $2M$ enthält höchstens 10 Punkte.

Satz 2.12: Laufzeit FindClosestPair

Der minimale Abstand aller Paare einer n-elementigen Punktmenge in der Ebene lässt sich in Zeit $\mathcal{O}(n \log(n))$ bestimmen.

2.3.2 Schnittpunkte von Strecken

Problem 2.13: Line Segment Intersection in zwei Dimensionen

Input: n Liniensegmente $s_i = l_i r_i$ in der Ebene für $i \in \{1, \ldots, n\}$
Output: Berechne die *echten Schnittpunkte*, also Punkte, die zum relativen Inneren beider Strecken bestehen

Wir nehmen an, dass keine Mehrfachschnittpunkte vorliegen und nie gleiche x-Koordinaten an beiden Segmentenden vorkommen. Außerdem sollen die geraden in *allgemeiner Lage* liegen, sie weisen also keinerlei geometrische Besonderheiten wie Parallelität oder Orthogonalität auf.

Im *Existenzproblem* fragen wir uns, ob es unter den n Strecken mindestens zwei gibt, die einen echten Schittpunkt haben, bein *Aufzählungsproblem* wollen wir *alle* echten Schnittpunkte bestimmen.

Wir fragen uns wieder nach einer sinnvollen Wahl von Ereignisstruktur und Sweep-Status-Struktur: Die ES bestehe aus den linken und rechten Endpunkten l_i und r_i, insbesondere deren x-Koordinaten. Als SSS wählen wir die Segmente gemäß einer noch festzulegenden Ordnung. Außerdem gibt es drei Typen von Ereignissen, die auftreten können:

1. Neuer linker Endpunkt eines Segments gefunden (*LeftEnd*),
2. Neuer rechter Endpunkt eines Segments gefunden (*RightEnd*),
3. Neuer Schnittpunkt gefunden (*Intersect*).

Lemma 2.14: Echte Schnittpunkte sind Nachbarn

Haben zwei Liniensegmente einen echten Schnittpunkt zum Zeitpunkt t, dann sind die Segmente unmittelbar vor t Nachbarin in der SSS geworden.

wir fragen uns jetzt, welche Operationen wir für die Lösung des *Existenzproblems* unterstützen müssen:

- Insert(SSS, seg, x) fügt Segment seg entsprechend der Ordnung zur Zeit x in die SSS ein.
- Delete(SSS, seg, x) entfernt Segment seg zur Zeit x aus der SSS.
- Prec(SSS, seg, x) bestimmt den Vorgänger von seg in der SSS zum Zeitpunkt x.
- Succ(SSS, seg, x) bestimmt den Nachfolger von seg in der SSS zum Zeitpunkt x.
- IntersectTest(seg_1, seg_2) testet auf Schnitt zwischen seg_1 und seg_2.

Für die Implementierun der SSS verwenden wir wieder einen AVL-Baum als balancierten Suchbaum, also ist jede der Operationen in $\mathcal{O}(\log(n))$ ausführbar, weil höchstens n Segmente in der SSS gespeichert sind und weil der Suchbaum balanciert ist. Da die Bearbeitung von jedem

der höchstens $2n$ vielen Ereignisse nur drei solche Operationen erfordert, erhalten wir folgende Aussage:

Satz 2.15: Existenzproblem Line Segment Intersection

Man kann in optimaler Zeit $\mathcal{O}(n \log(n))$ und mit Speicherplatz $\mathcal{O}(n)$ herausfinden, ob von n Strecken in der Ebene mindestens zwei einen echten Schnittpunkt haben.

Dabei haben Insert und Delete wieder Laufzeit $\mathcal{O}(\log(n))$, während IntersectTest nur $\mathcal{O}(1)$ benötigt.

Das Lösen des *Aufzählungsproblem* ist etwas anspruchsvoller. Nach einem Schnittpunkt wechselt die Ordnung zwischen zwei Segmenten, das vorher obere ist dann das neue untere Segment. Die konkreten Instanzen der drei oben beschriebenen Typen von Ereignissen, die während des Sweep zu behandeln sind, sind nicht alle schon von Anfang bekannt, deswegen brauchen wir eine *dynamische* ES. Außer Initialisierung und dem Test, ob ES leer ist, sind folgende Operationen möglich:

- Insert(ES, *Event*) fügt das Ereignis *Event* entsprechend seiner Eintrittszeit in die ES ein.
- NextEvent liefert das zeitlich erste in ES gespeicherte Ereignis und entfernt es aus der ES.

Dafür werden wir eine *Priority Queue* verwenden.

Zusammen haben wir folgenden Algorithmus:

Algorithmus 2.16 FindAllIntersections-2D

```
1:  Initialisiere SSS und ES
2:  Sortiere die 2n Endpunkte nach aufsteigenden x-Koordinaten
3:  Erzeuge daraus Events und füge diese in die ES ein
4:  while ES ≠ ∅ do
5:      Event ← NextEvent(ES)
6:      if Event is LeftEnd then
7:          Insert(SSS, seg, t)
8:          prevSeg ← Prec(SSS, seg, t)
9:          IntersectTestCreateEvent(prevSeg, seg)
10:         nextSeg ← Succ(SSS, seg, t)
11:         IntersectTestCreateEvent(seg, nextSeg)
12:     else
13:         if Event is RightEnd then
14:             prevSeg ← Prec(SSS, seg, t)
15:             nextSeg ← Succ(SSS, seg, t)
16:             Delete(SSS, seg, t)
17:             IntersectTestCreateEvent(prevSeg, nextSeg)
18:         else
19:             (uSeg, lSeg) ← (vorher oberes Segment, vorher unteres Segment)
20:             Nehme (uSeg, lSeg) als Paar mit Schnitt auf
21:             prevSeg ← Prec(SSS, uSeg, t)
22:             IntersectTestCreateEvent(prevSeg, uSeg)
23:             nextSeg ← Succ(SSS, uSeg, t)
24:             IntersectTestCreateEvent(lSeg, nextSeg)
```

Satz 2.17: FindAllIntersections-2D

Das Sweep-Verfahren berechnet die k echten Schnittpunkte von n Liniensegmenten in Zeit $\mathcal{O}((n+k)\log(n))$ mit Platz $\mathcal{O}(k+n)$.

Beweis. Insgesamt gibt es $2n + k$ Ereignisse. Weil keines von ihnen mehrfach in der Ereignis-Struktur vorkommt (da jeder Versuch, ein schon vorhandenes Element erneut einzufügen, von der Datenstruktur abgewiesen wird), sind niemals mehr als $\mathcal{O}(n^2)$ Ereignisse in der ES gespeichert. Jeder Zugriff auf die ES ist also in Zeit $\mathcal{O}(\log(n^2)) = \mathcal{O}(\log(n))$ ausführbar.

Die **while**-Schleife wird $(2n+k)$-mal durchlaufen; bei jedem Durchlauf werden höchstens sieben Operationen auf der SSS oder der ES ausgeführt. Daraus folgt die Behauptung. □

Satz 2.18: FindAllIntersections-2D weniger Platzbedarf

Das Sweep-Verfahren berechnet die k echten Schnittpunkte von n Liniensegmenten in Zeit $\mathcal{O}((n+k)\log(n))$ mit Platz $\mathcal{O}(k+n)$.

Wir können also auch degenerierte Fälle behandeln.

[16.04.2026, Vorlesung 2]
[21.04.2026, Vorlesung 3]

2.3.3 Einschub – Untere Schranken

Wir bekommen für Closest Pair in 2D einen $\mathcal{O}(n\log(n))$ Sweep-Algorithmus. Für den Schnitt von Liniensegmenten bekommen wir Sweep-Algorithmen mit $\mathcal{O}(n\log(n))$ für das Existenzproblem und $\mathcal{O}((n+k)\log(n))$ für das Aufzählungsproblem. Für die Frage, ob es bessere Algorithmen gibt, wollen wir Lower Bounds erarbeiten, dafür folgt jetzt etwas Technik.

Außerdem wollen wir den Begriff der *allgemeinen Lage* besser definieren, um *degenerierte Fälle* behandeln, beispielsweise Mehrfachschnittpunkte. Dabei können wir entweder *intrinsisch*, also *im* Algorithmus stattfinden (beispielsweise durch *lexikographische Ordnung*, also Sortierung nach x-Koordinaten, und bei Gleichheit weiter nach y-Koordinaten); oder wir arbeiten *extrinsisch*, das heißt, wir manipulieren die Eingabe.

Wir erinnern uns an die Landau-Symbole. Es ist $g \in \Omega(f)$ genau dann, wenn ein $n_0 \geq 0$ und $c > 0$ existieren mit $g(n) \geq c \cdot f(n)$ für alle $n \geq n_0$.

Für einen Algorithmus A, ein Problem Π und $P \in \Pi$ definieren wir $T_A(n) \coloneqq \max\{T_A(P) \mid |P| = n\}$ als die Anzahl wesentlicher Opertionen.

Für die Sortierung von n paarweise verschiedenen Objekten $q_1, \ldots, q_n$ („in allgemeiner Lage“), die eine *totale Ordnung* besitzen. Wir sagen

$$K(i,j) = \begin{cases} 1 & q_i < q_j \\ -1 & q_j < q_i, \end{cases}$$

diese Operation (*Schlüsselvergleich*) benötigt $\mathcal{O}(1)$ Operationen. Wir bestimmen eine Permutation $\pi : \{1,\ldots,n\} \xrightarrow{\sim} \{1,\ldots,n\}$, sodass $q_{\pi(1)} < q_{\pi(2)} < \cdots < q_{\pi(n)}$ gilt.

Satz 2.19: Sortieren durch Schlüsselvergleiche

Sortieren durch Schlüsselvergleiche hat Komplexität $\Omega(n\log(n))$.

Beweis. Sei A ein deterministischer Algorithmus zur Sortierung durch Schlüsselvergleiche. Wir definieren $W(i,j) \coloneqq \{(q_1,\ldots,q_n) \in Q^n \mid q_i < q_j\}$. Alle Eingaben, die dieser Menge angehören, erfüllen $K(i,j) = 1$. Wird also $K(i,j)$ als erster Aufruf der Vergleichsfunktion berechnet, folgt darauf die Folge der weiteren Vergleiche. Dann erhalten wir einen *Entscheidungsbaum* von A als binären Baum, dessen Knoten mit Vergleichen „$q_i < q_j$?“ beschriftet sind. An den Kanten steht entweder 1 oder 0, je nach Wahrheitsgehalt. Weil A das Sortierproblem löst, gibt es zu jedem

Blatt des Entscheidungsbaums eine Permutation π, sodass nur Inputfolgen mit

$$q_{\pi(1)} < q_{\pi(2)} < \cdots < q_{\pi(n)}$$

zu diesem Blatt führen. Für jede Permutation muss ein Blatt vorhanden sein, also hat der Entscheidungsbaum mindestens $n!$ Blätter und somit mindestens die Höhe

$$\log_2(n!) \geq \log_2\left(\left(\frac{n}{2}\right)^{\frac{n}{2}}\right) = \frac{n}{2}\log_2\left(\frac{n}{2}\right) \geq \frac{1}{3}n\log_2(n) \quad \forall n \geq 8.$$

Es gibt also eine Permutation, für die das Verfahren A mindestens $\frac{1}{3}n\log_2(n) \in \Omega(n\log(n))$ viele Vergleiche ausführt. □

Da Heapsort oder Mergesort in $\mathcal{O}(n\log(n))$ sortieren, hat das Sortierproblem Komplexität $\Theta(n\log(n))$.

Problem 2.20: Lineares Modell / Elementtest

Input: $c_1, \ldots, c_n, d \in \mathbb{R}$, Punkt $(x_1, \ldots, x_n) \in \mathbb{R}^n$
Output: Ist $c_1x_1 + \cdots + c_nx_n + d < 0$?

Wir werten also eine lineare Funktion h aus $(\mathcal{O}(1))$ Beispielsweise für $c_i = 1, c_j = -1$ ist $h(x_1, x_2) = x_1 - x_2$. Diese Funktion beschreibt eine (Hyper-)Ebene im $\mathbb{R}^n$. Wir fragen uns nach einer unteren Schranke für den *Elementtest* bzw. sein Entscheidungsproblem: Für $W \subseteq \mathbb{R}^n$, liegt $x \in W$?

Satz 2.21: Elementtest auf Zusammenhangskomponenten

Sei $W \subseteq \mathbb{R}^n$ eine Menge mit m Zusammenhangskomponenten. Dann benötigt jeder Algorithmus für den Elementtest von W mindestens $\log_2(m)$ viele Schritte, das Problem liegt also in $\Omega(\log(m))$.

Ein Beispielproblem wäre beispielsweise binäre Suche für m disjunkte Teilintervalle von $\mathbb{R}$ zur Bestimmung, ob x in einem dieser Punkte liegt.

Beweis. Sei A ein Algorithmus für den Elementtest für W. Wie im Beweis von Satz 2.19 betrachten wir den linearen Entscheidungsbaum E von A, dessen Konten die Tests

$$h(x_1, \ldots, x_n) < 0?$$

in allen möglichen Rechenabläufen von A darstellen. Es genügt zu zeigen, dass E mindestens so viele Blätter hat wie die Menge W Zusammenhangskomponenten. Für jedes Blatt b von E sei

$$A_b = \{x \in \mathbb{R}^n \mid A \text{ terminiert bei Eingabe von } x \text{ im Blatt } b\}.$$

Dieses A_b ist – nach Definition des linearen Modells – Durchschnitt von offenen und abgeschlossenen affinen Teilräumen

$$T_h := \{(x_1, \ldots, x_n) \in \mathbb{R}^n \mid h(x_1, \ldots, x_n) < 0\}$$
$$\overline{T_h} := \{(x_1, \ldots, x_n) \in \mathbb{R}^n \mid h(x_1, \ldots, x_n) \geq 0\}.$$

Wir merken, dass T_h und $\overline{T_h}$ konvex sind, insbesondere also zusammenhängend. Für alle $x \in A_b$ trifft A dieselbe Entscheidung; folglich ist $A_b \subseteq W$ oder $A_b \cap W = \emptyset$. Da der Algirithmus für jede Eingabe x eine Entscheidung treffen muss, folgt

$$\mathbb{R}^n = \bigcup_{b \text{ Blatt}} A_b.$$

Damit erhalten wir

$$\begin{aligned} W &= \mathbb{R}^n \cap W \\ &= \bigcup_{b \text{ Blatt}} A_b \cap W \\ &= \bigcup_{b \in B} A_b \end{aligned}$$

für eine Teilmenge B aller Blätter von E. Die Anzahl der Zusammenhangskomponenten von dieser Menge ist also höchstens so groß wie $|B|$, der Anzahl der zusammenhängenden Teilmengnen A_b, weil jedes A_b vollständig in einer Zusammenhangskomponente enthalten ist. Andererseits ist $|B|$ höchstens so groß wie die Anzal *aller* Blätter von E. □

[21.04.2026, Vorlesung 3]

[23.04.2026, Vorlesung 4]

[23.04.2026, Vorlesung 4]

[28.04.2026, Vorlesung 5]

Anhang

Liste der Definitionen

2.1 Find maximum 2
2.3 Closest pair in einer Dimension 2
2.6 Maximum subvector 3
2.9 Closest pair in zwei Dimensionen 3
2.13 Line Segment Intersection in zwei Dimensionen 5
2.20 Lineares Modell / Elementtest 8

Liste der Aussagen

2.8 FindMaximumSubvector 3
2.11 Höchstens 10 Punkte in Rechteck 5
2.12 Laufzeit FindClosestPair 5
2.14 Echte Schnittpunkte sind Nachbarn 5
2.15 Existenzproblem Line Segment Intersection 6
2.17 FindAllIntersections-2D 6
2.18 FindAllIntersections-2D weniger Platzbedarf 7
2.19 Sortieren durch Schlüsselvergleiche 7
2.21 Elementtest auf Zusammenhangskomponenten 8

Liste der Algorithmen

2.2 FindMax 2
2.4 FindClosestPair-1D 3
2.7 FindMaximumSubvector 3
2.10 FindClosestPair-2D 4
2.16 FindAllIntersections-2D 6

Liste der Abkürzungen

AVL Adelson-Velsky, Landis . . . . . 4 f.
ES Ereignisstruktur 4–7
SSS Sweep-Status-Struktur . . . . . 4 f., 7

Index

allgemeine Lage, 5, 7
Aufzählungsproblem, 5
AVL, 4, 5

degeneriert, 7

ES, 4–7
Existenzproblem, 5

extrinsisch, 7

intrinsisch, 7

lexikographische Ordnung, 7

SSS, 4, 5, 7
Sweep, 2